Fresh Eyes: Clinical Agents Miss Orders They Already Know, and a One-Pass Model That Reads Only the Task Finds Them

Yubo Li **Rema Krishnan**
Carnegie Mellon University
yubol@andrew.cmu.edu

Abstract

LLM agents working in the electronic health record finish consultations with required orders unplaced. Most are not gaps in knowledge: across five logged runs of each task the agent proposes 91% of the required orders but 75% in a single run, and given nothing but the task text it names 84% of the orders it went on to miss. In tests on two agents from different model families (Qwen3.8-27B and Gemma 4 31B), mostly pre-registered, two mechanisms turn knowledge into omission: agents order before they have written a plan, and they anchor on the orders they have already placed. The anchor extends to anything that reads the agent's work. Six System One decision models and the LLM itself rank the missed orders *below chance* when they read the agent's trajectory (AUROC 0.37–0.46) and above chance when they read only the task (up to 0.73). The agent's orders, not its notes, are the anchor, and telling the checker to look for omissions does not help. The fix is fresh eyes: a System One model reads only the task and ranks candidate orders in under a second each. Shown when the agent first tries to finish, its eight-item checklist raises the pass rate of the required-order checkpoints still failing at that point from 38% to 53% for Qwen and from 29% to 58% for Gemma, beats a random checklist from the same pool, and adds orders that blinded LLM judges rate mostly as indicated or reasonable, none as potentially harmful.

1 Introduction

A physician who has just finished a workup is the person least likely to spot what is missing from it. Medicine answers this with second readers, consults and checklists: a fresh pair of eyes that has not followed the same path (Croskerry, 2003; Haynes et al., 2009). LLM agents now do physicians' work in the electronic health record. They look up the problem list, pull the labs, order the workup and write the note (Liu et al., 2026; Yao et al., 2023). We find that they miss orders for the same reason, and that the fresh-eyes remedy works for them too, provided the second reader really has fresh eyes.

We began with a different question. *System One* decision models, named after the fast mode of human judgment (Kahneman, 2011), read a state and a set of options and return a probability for each option in a single forward pass (Almeida, 2026; Cloudflare, 2026). They are fast and cheap, and choosing among options is what an agent does at every step. So we put six of them in the decision seat of a clinical agent on PhysicianBench, 100 real consultations on a running FHIR server. They are safe there, and they do not help (§3). Following every required order along the agent's trajectories shows why: only 11% of required orders are ever proposed and then not executed, so no selector can recover much, while 19% are never proposed at all. The orders are not lost when the agent chooses among its options. They are lost before that, when the agent generates them.

That moved the question from *who decides* to *why the agent never proposes what it needs*. Three findings answer it.

1. The agent knows the orders it misses (§4). The required orders a run never proposes are mostly proposed in other logged runs of the same task (91% coverage over five runs against 75% in one). Asked with nothing but the task text, the same LLM names 84% of the orders its run left open. This is a knowing–doing gap, not a knowledge gap.

2. It plans late and anchors on its own orders (§5). At its first order step, asking the agent to write out its full plan before ordering recovers orders it would otherwise miss for good; repeating the task does not. At its finish, asking what is still missing recovers more when the agent's own orders and notes are removed from its context. Both hold on



Figure 1: **The paper in one figure.** *Left:* a clinical agent finishes with a required order unplaced, although the same model proposes it in other runs and names it from the task text alone (§4). It plans late and anchors on its own orders (§5). *Middle:* a checker that reads the agent's trajectory, whether a System One model or the LLM, scores each candidate order largely by its resemblance to what the agent already ordered, so the missed orders land below chance; reading only the task, the same models rank them above chance (mean over six System One models; §6). *Right:* a System One model that reads only the task ranks a checklist that, shown at the agent's finish, recovers the missed orders for both agents (§7). All numbers are aggregates; no patient data.

two agents from different model families, Qwen3.8-27B and Gemma 4 31B (anchoring pre-registered on both, planning on Gemma), and the effects are larger on the weaker agent.

3. Any checker that reads the agent's work inherits its blind spot (§6). The obvious remedy for omissions is a second model that reviews the agent's work. Six System One models and the thinking LLM itself, reading the agent's trajectory, rank the orders the agent missed below the unneeded ones. Given the task text alone, all of them rank the missed orders above. For every System One model we tested, the anchor is the agent's orders, not its notes or the chart it retrieved. One fabricated order shifts the judgment causally. Telling the checker that the agent may have missed something does not help. The checker scores an order by how much it resembles what the agent already did.

The fix: fresh eyes (§7). The three findings point to one design. Give the checker the task, not the agent's work. Let a System One model rank a broad pool of candidate orders, which is cheap because it decides in one forward pass, and leave the decision to the agent. At the agent's finish, an eight-item checklist ranked by a System One model from the task text alone raises the pass rate of the still-open required-order checkpoints from 38% to 53% for Qwen and from 29% to 58% for Gemma. It beats a random checklist from the same pool, holds with a second System One ranker, and does as well as restarting the agent from a fresh context. The orders it adds beyond the rubric are, by blinded LLM judges, mostly indicated or reasonable, and none is rated potentially harmful.

How we tested. Whole agent runs vary a lot. Two runs of the same agent differ on 28 of 100 tasks, and an early online gain of ours did not survive independent replication. We therefore pre-registered every confirmatory test, with its prediction and bar, before its data existed (Appendix C). We also measure online interventions by *branching*: every arm replays the same logged run up to the agent's first finish and differs only from there, so that run-to-run noise cancels.

2 Setting

PhysicianBench (Liu et al., 2026) adapts 100 real outpatient consultations from 21 specialties into long-horizon tasks on a FHIR server holding the records of 108 de-identified patients. Each task has 5–9 checkpoints (670 in total) of four types (data retrieval, clinical reasoning, action execution and documentation), each graded by code or by an LLM judge. We use the released harness unchanged: a fresh server per task, the MiniAgent loop with parallel tool calls and at most 100 steps, and the task's own graders. Our focus is the 92 *required-order checkpoints* in 54 tasks: code-graded checks that a specific laboratory test, imag-

ing study, referral, procedure or medication was placed in the record. Some checkpoints require several orders, 118 in all. We match an order to such a checkpoint with the grader's own name and code patterns. An order that matches none we call *outside the rubric*.

Agents. The main agent LLM is Qwen3.8-27B (Qwen Team, 2026) with thinking on, in bf16 with 128k context. The second agent is Gemma 4 31B-it, a different model family, with thinking on and FP8 weights. FP8 is the precision at which one 128k context fits beside the judge on our GPUs. As an agent it is much weaker on this benchmark: it passes 51% of checkpoints against Qwen's 81% and 8% of tasks against 41%, and it places no order in 56% of its runs. Both run in the same harness, and the grader's judge is Qwen with thinking off for every arm.

System One models. Six open decision models take a state, a typed question and named options and return a probability per option in one forward pass: Clef-27B (Cloudflare, 2026) (post-trained from the same Qwen backbone), pplx-decider-v1.2 (pplx12), d3, linne-oak-1.0, amos-1-large and decider-31b, each served with its authors' code at a pinned revision (Appendix A). Everything runs locally on one node; no record leaves it (§9).

Branching from a shared prefix. For an intervention at the agent's finish we take each logged run whose first FINISH leaves a required-order checkpoint *open* (still failing): 51 cases for Qwen and 65 for Gemma. Every arm replays that run's turns against a fresh environment up to the FINISH and branches there. The *none* arm keeps the logged finish; replaying it reproduces 99.7% (Qwen) and 99.6% (Gemma) of the logged checkpoint outcomes. The other arms run 3–5 times each. The primary outcome is the pass rate of the checkpoints that were open at the branch. We also report orders placed after the branch, those outside the rubric, and forbidden ones (none occurred). Differences are paired over cases with 95% bootstrap intervals.

3 The decision seat is not where orders are lost

Our starting point was to let a System One model make the agent's decisions (Appendix D gives the full study). In S1-SELECT, the LLM proposes five complete next turns and the System One model picks one. In S1-TOOL, the System One model picks the kind of action and the LLM writes the call. Choosing among proposals, Clef keeps the LLM's level (80.2% of checkpoints against 81.3%; task success -4 points, $p = 0.52$). Choosing the action itself, it breaks the agent (7% of tasks pass), because it acts after little retrieval and rewrites the note again and again. All six System One models share a *commitment bias*: on a fixed exam of 1,460 logged decisions they pick ordering turns 1.3–1.6× and finishing turns 1.3–1.4× as often as the LLM does.

Required orders (%)	LLM	Clef	pplx12
Placed	69	64	72
Proposed, never executed	11	8	4
Never proposed	19	28	24
Step omissions placed later	79	13	25

Table 1: **Where required orders are lost,** along each decision maker's own trajectories (54 tasks, 118 required orders; Clef and pplx12 in S1-SELECT). The middle row is the most any selector could recover; the agent's real loss is the orders it never proposes.

Neither seat improves on the LLM, and Table 1 shows why. Of the 118 required orders in 54 tasks, the LLM places 69%. Only 11% are proposed at some step and never executed, which is all a selector could ever recover. 19% are never proposed by any of the five samples at any step. Even the step-level omissions are mostly a matter of timing: 79% of the required orders the LLM's sample skips at an order step are placed later in the same run. The pre-registered tests agree. The best decider's early gain on required orders failed replication on unseen tasks, and a decider consulted only at order steps adds orders, mostly outside the rubric, without adding required ones. The errors that matter happen before the decision, at generation. The rest of the paper asks why the agent never proposes orders it needs.

4 The agent knows the orders it misses

If the agent lacked the knowledge, no amount of re-reading would help. It does not lack it. We pooled the orders proposed by any sample in five logged runs of each task: the LLM's own run and four runs in which a System One model took part in its decisions (§3). On average a single run proposes 75% of the required orders and places 68%; the five runs together propose 91% (Figure 2a). Of the required orders a run never proposes, 62% are proposed in another run of the same task. A second test asks

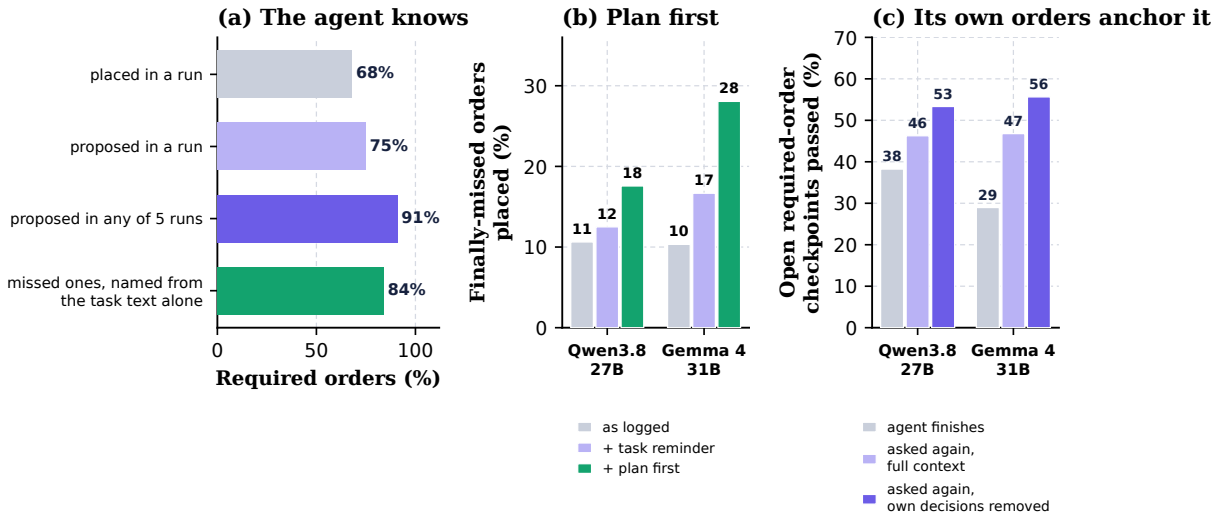


Figure 2: **Why the agent misses orders.** (a) Qwen3.8-27B knows the orders it misses: share of required orders placed and proposed in a single run (mean over five logged runs), proposed in any of the five (54 tasks, 590 order $\times$ run pairs), and the share of the orders its run left open that Qwen names from the task text alone (26 tasks, 36 orders). (b) It plans late: at the first order step, replayed with $K{=}5$ samples, the share of the orders the run finally missed that the agent places, as logged, after a reminder of the task, and after being asked to write its full plan first (Qwen 48 cases, Gemma 18; pre-registered for Gemma; for Qwen the pre-registered test covered all required orders at that step, +19 points). (c) It anchors on its own orders: at its first finish, the share of open required-order checkpoints that pass when the agent finishes as logged, when it is asked for the missing orders with its full context, and when it is asked with its own decision turns removed (Qwen 51 cases $\times$ 5 runs, Gemma 65 cases $\times$ 3 runs; pre-registered).

directly. Qwen, without thinking and given nothing but the task text, lists the orders that could be indicated, eight samples per task. Its list covers 84% [71, 95] of the required orders that the agent's own run left open at its finish. The model that failed to place these orders while working in the chart names them readily from the request alone. What fails is the step from knowing to doing.

5 Why it misses them: it plans late, and it anchors on its own orders

Acting before the plan is written. One explanation is drift: over a long trajectory of record lookups the agent loses track of the task. Another is that it starts ordering before it has enumerated what the task needs, and what it never enumerates it never orders. Drift predicts that a reminder helps; late planning predicts that a plan does. We replayed the agent's first order step with five samples under three conditions: as logged, with the task instruction repeated, and with a request to write the full assessment and plan, listing every order the task requires, before placing any (Figure 2b). For Qwen the reminder moves orders earlier (pre-registered, +11 points of all required orders at that step) but not into the final record; for Gemma it does neither significantly. Planning first does both. For Qwen, the share of the run's finally missed orders placed at that step rises from 11% to 18% (+7 points [2, 13]), against +2 [-2, +5] for the reminder. For Gemma, where we pre-registered this comparison on the finally missed orders, it rises from 10% to 28% (+18 [4, 32]) and beats the reminder by +11 [2, 23]. Forgetting the task affects when orders are placed; the orders lost for good are those the agent commits past before its plan exists.

Anchoring on its own orders. At the agent's first finish we asked for the missing orders in two ways (Figure 2c). Both arms ask the same question, "place every order that this task requires and that has not been placed yet". One asks it with the agent's full context. The other asks it with every decision turn removed: no orders, messages or notes, only the task and the chart the agent retrieved. Repeats of orders already placed are filtered out in both. Asking at all helps: open required checkpoints rise from 38% to 46% for Qwen (+8.0 [2.6, 15.1]) and from 29% to 47% for Gemma (+17.8 [10.6, 25.7]), so the agent can still produce the orders. Asking without its own decisions in view helps more: +7.1 [1.0, 13.7] points over the full context for Qwen, and +8.9 [3.2, 15.3] for Gemma, and the pass rate

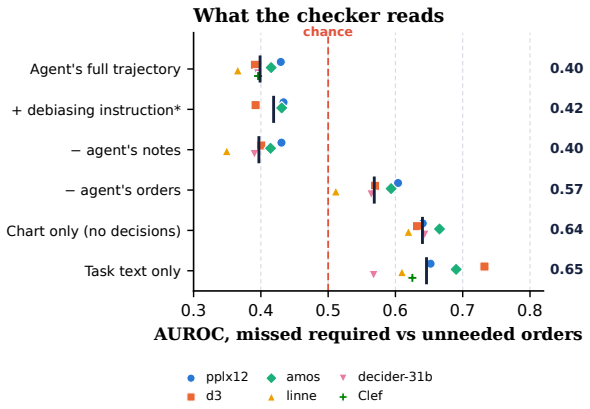


Figure 3: **A checker that reads the agent inherits its blind spot.** AUROC of each System One model for telling the required orders the agent missed from unneeded candidate orders (21 tasks with a missed required order), by what the checker reads: the agent's trajectory at its finish; the same with a debiasing instruction (*three models); without the agent's notes; without its orders; the retrieved chart with no decisions; the task text alone. Black line: mean over the models shown in the row.

over all checkpoints does not drop. The agent's record of what it has done suppresses orders that it can still generate. Both comparisons were pre-registered for both agents, and the effects are larger for the weaker one.

Beyond orders. The agents differ in their other failures. Labelled by the local LLM, Qwen's failed clinical-reasoning and documentation checkpoints are most often *wrong* content (wrong judgments in reasoning, 70% of the wrong items; misstated facts in notes, 62%), while Gemma's are most often *incomplete*. When Gemma's note mentions an order it never placed, 60% of the time it is a recommendation it did not carry out (Qwen: 10%). The error profile depends on the agent. The two mechanisms behind missed orders, and the remedies that follow from them, hold for both.

6 A checker that reads the agent inherits its blind spot

The standard remedy for an agent's omissions is review: a second model reads the agent's work and flags what is missing, as in self-refinement and LLM-as-judge pipelines (Madaan et al., 2023; Zheng et al., 2023). A System One model is an attractive reviewer here. It scores a candidate order for a patient in one forward pass, so it can screen dozens of candidates where an LLM would deliberate over a few. We tested this offline at the agent's first finish. The candidates are every order proposed in the task's five runs that the agent has not placed (about 44 per task). Each checker scores each candidate with the question "is this order indicated for this patient as part of completing the task?". The measure is how well it separates the required orders the agent missed from the unneeded ones (AUROC, per task).

Reading the trajectory, every checker ranks the missed orders below chance. With the agent's trajectory as input, all six System One models score 0.37–0.43 (Figure 3). Given only the task text, the same models score 0.57–0.73, a paired gain of +0.17 to +0.34 with every interval excluding 0 (pre-registered). The LLM is no exception. In an exploratory control, Qwen with thinking goes from 0.46 to 0.62 on the same questions (+0.17 [0.07, 0.27]). The anchoring is what happens to any judge that reads the agent's actions, not a quirk of small decision models. The trajectory hurts even on the orders the agent did place (0.65–0.72 with it, 0.74–0.86 without).

The anchor is the agent's orders. Removing only the agent's decision turns (orders, messages, notes) and keeping the full chart it retrieved brings the checker back to about its task-only level (0.62–0.67; +0.21 to +0.25 over the trajectory, five of five models; pre-registered). The chart does not mislead the checker; the agent's decisions do. Among the decisions, the orders carry 57–83% of the recovery and the notes none (−0.07 to +0.04). A causal test confirms the direction. Inserting one fabricated step in which the agent places the candidate order raises the checker's P(Yes) by +0.07 to +0.10, for required and unneeded orders alike.

The mechanism is plan conformity. With the trajectory, a checker's score tracks how closely the candidate resembles the agent's placed orders (within-task Spearman 0.28–0.38 for System One models, 0.47 for Qwen; 0.05 with the task text). The missed orders resemble the agent's placed orders less than unneeded candidates do (name overlap 0.26 against 0.34), so similarity alone ranks them below chance (AUROC 0.41). A checker that reads the agent's work grades candidates by their conformity to that work, so it inherits exactly the agent's omissions.

An instruction does not fix it; the input must change. Adding "the agent may have missed orders; judge each one independently" recovers 1–

6% of the gap for System One models and 8% for Qwen (pre-registered). The models read the sentence, since their P(Yes) rises across the board, but the ranking does not change.

Fresh eyes are cheap for a System One model. Given the task text alone, a System One judgment takes 0.2–0.6 s and always yields a score. Qwen with thinking takes 41–69 s per judgment and leaves 17–31% unanswered within an 8k-token budget, and on the questions it does answer, task-only System One models match or beat it. Screening dozens of candidate orders per task costs seconds with a one-pass model.

7 Fresh eyes: a System One checklist from the task

The findings fix the design. The checker reads the task, not the agent's work (§6). Candidates come from a broad pool, because the agent knows the orders but does not reach them in one pass (§4). The intervention comes when the agent's own plan is complete and it is about to stop (§5). The agent keeps the decision.

Design. Before the run, the LLM without thinking reads the task text and lists candidate orders: eight samples of up to 15 each, merged by name, about 67 per task. A System One model scores every candidate from the task text alone. At the agent's first finish, candidates already placed are dropped: by exact name for Gemma, and for Qwen also the variants that pplx12, reading only the placed orders, judges repeats of one. The top eight are shown with one instruction: "Check each against the chart and the task. Place any that are indicated, and only those, then finish." (Appendix B). The System One cost is one forward pass per candidate, under a second each, and the ranking is computed before the run starts.

It recovers the missed orders, for both agents. From the same prefix, the checklist raises open required-order checkpoints from 38% to 53% for Qwen (**+14.7** points [6.7, 24.1]) and from 29% to 58% for Gemma (**+29.2** [20.0, 39.5]) (Figure 4a). The ranking matters. A random eight from the same pool does less, and the System One list beats it by +5.9 [0.6, 11.8] for Qwen and +13.6 [6.5, 21.7] for Gemma. A different System One ranker, d3, does as well as pplx12 for Qwen (54%; +16.0 [7.8, 25.2] over the agent's finish, +7.2 [1.1, 13.3] over the random list). The effect belongs to System One ranking of a task prior, not to one model. All of these were pre-registered, and all hold. The checklist does as well as the fresh-context restart of §5 (-0.4 points for Qwen, -2.6 for Gemma, both n.s.): two routes to the same fix, changing what the agent reads before it stops. Over all checkpoints, not only the open ones, the pass rate rises by 4.7 points for Qwen and 7.8 for Gemma.

The extra orders are reasonable care, not noise. The checklist also adds about 1.5 orders per case that no rubric item names (+1.47 for Qwen, +1.56 for Gemma). The random list adds more (+2.0, +2.3). PhysicianBench neither credits nor penalises such orders, so we asked what they are. A blinded LLM judge saw the task and the retrieved chart and rated each order alone, without its source, as indicated, reasonable, low value or potentially harmful: Gemma 4 rated every order, Qwen a random sample of 150 per agent. (The pre-registered version rated each case's orders in one list, where near-duplicate variants sat side by side; it agrees on harm and is mixed on the rest, Appendix C.) By the Gemma judge (Figure 4b), 81% of the checklist's extra orders for the Qwen agent and 91% for the Gemma agent are indicated or reasonable. That is at least the rate of the agents' own orders outside the rubric (78%, 80%), and well above the random list's (67%, 80%; difference in inappropriate share -0.14 [-0.22, -0.06] and -0.11 [-0.18, -0.05]). None was rated potentially harmful, against 2.9% and 3.9% of the agents' own extra orders. The Qwen judge agrees in direction (inappropriate share, checklist against random list: -0.31 and -0.36, both intervals excluding 0; κ 0.48–0.49 between judges). For the Qwen agent's cases both judges also rate a fifth of the *required* orders low value, so those shares are best read side by side. What the rubric counts as excess is mostly care it does not list.

8 Related work

Self-correction and review. Agents and LLMs are often asked to critique and revise their own output (Madaan et al., 2023; Shinn et al., 2023), and reasoning errors are hard to self-correct without external signal (Huang et al., 2024). LLM judges favour their own generations (Panickssery et al., 2024). We locate a specific failure of review inside an agent loop. A reviewer that reads the agent's work, whether it is the agent itself, another LLM or a System One model, scores candidates by their

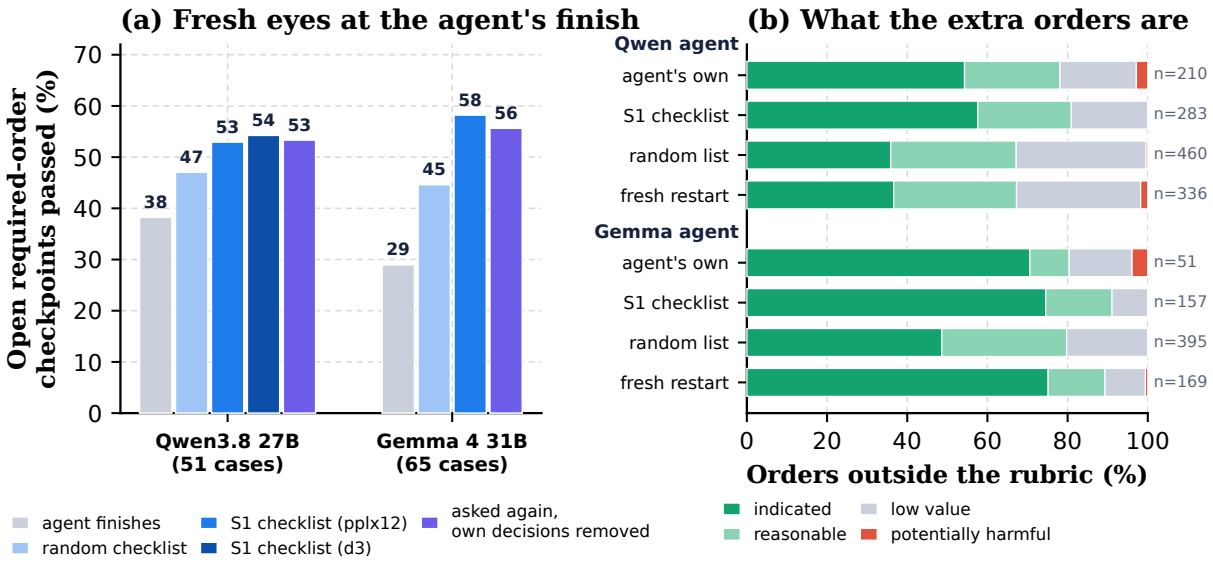


Figure 4: **Fresh eyes in the loop.** (a) At each agent's first finish, from the same replayed prefix: the share of open required-order checkpoints that pass when the agent finishes as logged; when it sees eight orders drawn at random from the task-prior pool; when it sees the eight ranked highest from the task text by pplx12 (both agents) or d3 (Qwen only); and, for reference, the fresh-context restart of Figure 2c. Qwen: 51 cases × 5 runs (d3: 3); Gemma: 65 cases × 3 runs. All comparisons with *agent finishes* and with the random list were pre-registered. (b) What the orders outside the rubric are: each order rated alone by a blinded LLM judge (Gemma 4) that sees the task and the retrieved chart. Bars give the share of distinct orders per case by source; n is the number of orders rated.

conformity to that work. Changing what the reviewer reads fixes it, and instructions do not.

Anchoring and premature closure. Anchoring on an initial estimate (Tversky and Kahneman, 1974) and premature closure are among the most cited cognitive errors in clinical diagnosis, and fresh perspectives and checklists are the classic remedies (Croskerry, 2003; Haynes et al., 2009). We find the agent-level analogue: the agent anchors on the orders it has placed, and so does anything that reads them. Planning before acting (Wang et al., 2023a) also helps here, though it does not close the gap.

Choosing among samples. Self-consistency (Wang et al., 2023b), trained verifiers (Cobbe et al., 2021) and process reward models (Lightman et al., 2024) pick among sampled outputs. Our S1-SELECT applies this at every agent step with a System One model as the selector. Our ceiling analysis shows why selection has little to gain when the missing content is never sampled.

System One models and clinical agents. JEV introduced the typed-question interface that System One models answer in one forward pass (Almeida, 2026); Clef is an open 27B model of this kind (Cloudflare, 2026). They approach frontier LLMs as judges (Li et al., 2026b), and as selectors in general agents most of their wrong actions follow a correct reading of the state (Li et al., 2026a). PhysicianBench (Liu et al., 2026) grades physician tasks in a live record system and has been run only with generative agents. We find that the useful place for a System One model in a clinical agent is beside the agent as a fast, task-reading screener, not in its decision seat.

9 Discussion

Design rules. Three rules follow, and none is specific to one model. *Check the task, not the agent:* a reviewer, human or model, should read the request and the chart, not the agent's decisions, because the decisions pull its judgment toward conformity. *Plan before acting, review before finishing:* the agent loses orders by committing before the plan exists and by stopping once its own orders look complete. *Use the one-pass model for breadth:* its value is in screening many candidates from the task in under a second each, while the LLM generates the candidates and keeps the decision.

Why the decision seat failed. A System One model in the decision seat can only choose among what the agent proposes, and the agent's costly misses are never proposed. Its commitment bias

may make this worse: it ends episodes earlier, and with them the generation that could have surfaced the missing orders. The same models succeed once they are moved off the trajectory and onto the task.

The anchor is general. The checker in §6 and the agent in §5 fail in the same way: both score what to do next by what has already been done. The agent's restart and the checker's input ablation are the same intervention, removing the agent's decisions from view, and they help both. We expect this to hold for agents that act in other record systems and other domains wherever omissions matter: software changes, audits, procurement.

Limitations

We study one benchmark of outpatient consultations with two agent LLMs; the second, in FP8, is much weaker, and all our tests compare arms within one agent. An LLM ranker without thinking could replace the System One ranker; we compared against the thinking LLM only. The rubric names one acceptable set of orders. Our judgments of orders outside it come from two LLM judges with moderate agreement, not physicians. The grader's judge is the local Qwen model shared by every arm, so absolute scores are not comparable with the leaderboard. Online effects are measured from shared prefixes at the agent's first finish. An end-to-end deployment that combines planning first with the checklist remains to be run.

Ethics and data

PhysicianBench's records are real de-identified patients under a data use agreement. All models run locally, a socket guard blocks any non-local connection from agent and grader processes, and trajectories, notes, candidate orders and judge ratings stay on access-controlled storage outside version control. We report aggregates only. The checklist is a screening aid shown to an agent that keeps the decision. In deployment, a clinician would review every order.

References

D. Almeida. 2026. Introducing system one models & jev. TypeSafe AI blog, https://typesafe.ai/blog/introducing-system-one-models-and-jev.

Cloudflare. 2026. Clef decision models. Blog post and model card, https://blog.cloudflare.com/clef-decision-models. Model: Cloudflare/clef (27B, post-trained from Qwen3.8-27B).

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *Preprint*, arXiv:2110.14168.

Pat Croskerry. 2003. The importance of cognitive errors in diagnosis and strategies to minimize them. *Academic Medicine*, 78(8):775–780.

Alex B. Haynes, Thomas G. Weiser, William R. Berry, Stuart R. Lipsitz, Abdel-Hadi S. Breizat, E. Patchen Dellinger, et al. 2009. A surgical safety checklist to reduce morbidity and mortality in a global population. *New England Journal of Medicine*, 360(5):491–499.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations*.

Daniel Kahneman. 2011. *Thinking, Fast and Slow*. Farrar, Straus and Giroux.

Yubo Li, Ramayya Krishnan, and Rema Padman. 2026a. Reading right, acting wrong: Diagnosing decision failures of system one models in agent loops. *Preprint*, arXiv:2610.18579.

Yubo Li, Yidi Miao, Ramayya Krishnan, et al. 2026b. JEV-as-a-judge: Accept when confident, escalate when unsure. *Preprint*, arXiv:2609.26550.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *International Conference on Learning Representations*.

Ruoqi Liu, Imran Q. Mohiuddin, Austin J. Schoeffler, Kavita Renduchintala, Ashwin Nayak, Prasantha L. Vemu, Shivam C. Vedak, Kameron C. Black, John L. Havlik, Isaac Ogunmola, Stephen P. Ma, Roopa Dhatt, and Jonathan H. Chen. 2026. Physicianbench: Evaluating llm agents on physician tasks in real-world ehr environments.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*.

Arjun Panickssery, Samuel R. Bowman, and Shi Feng. 2024. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems*.

Qwen Team. 2026. Qwen3.8-27B. Model card, https://huggingface.co/Qwen/Qwen3.8-27B.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*.

Amos Tversky and Daniel Kahneman. 1974. Judgment under uncertainty: Heuristics and biases. *Science*, 185(4157):1124–1131.

Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023a. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023b. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.

A Decision interface

Questions. The System One model receives the state and one Choice question. For S1-SELECT: *"The state shows a clinical task assigned to an AI agent that works in an electronic health record through FHIR tools, followed by the agent's actions and their results so far. Each option is a candidate next action for the agent. Choose the candidate that best advances completing the task correctly, completely and safely."* For S1-TOOL the second and third sentences read: *"Each option is a tool the agent can use for its next action, or finishing with a final answer. Choose what the agent should do next to complete the task correctly, completely and safely."* Options are the rendered turns (S1-SELECT) or each tool's name with the first paragraph of its description, plus FINISH (S1-TOOL).

State. The state is the task instruction followed by every step: the action, rendered as tool calls with full arguments, and its results as the LLM saw them. It is fitted into the System One model's 32,768-token context deterministically: tool results are kept in full, then shortened oldest-first to 800 characters, then omitted oldest-first, each by the smallest number that fits. In the S1-SELECT run 44% of decisions saw shortened results and 1% saw omitted ones.

Other System One models. pplx-decider-v1.2 (`cce0f3f8`), d3 v3.1.0 (`cbff3a3c`), linne-oak-1.0 (`20da7bf3`), amos-1-large (`d67e051c`) and decider-31b (`e12ddf20`) each run their authors' own serving code on H100 GPUs, locally and offline. pplx-decider-v1.2 ships a library only, which we call on its reference path. decider-31b is NVFP4, which an H100 runs through vLLM's weight-only FP4 path; its authors used Blackwell GPUs. linne-oak-1.0 needs two GPUs for 32k-token inputs. Every model receives the state and question built for Clef, with the same fitting rule, counted in its own tokenizer. Five exam questions whose options alone exceed the 32k window are unanswerable for every model.

The LLM. Qwen3.8-27B with thinking on, temperature 1.0, top-p 0.95, top-k 20, at most 16,384 completion tokens per call. Identical replicas serve all arms; each task is routed whole to the less loaded replica, independent of the arm.

B Prompts

Task-prior pool (LLM, no thinking, eight samples). *"A clinical task assigned to an AI agent that works in a patient's electronic health record: [task]. Without seeing the chart, list the orders (laboratory tests, imaging, referrals or consults, procedures, medications) that could be indicated as part of completing this task. Up to 15, one per line, each in the form ServiceRequest: <name> (code <code>) or MedicationRequest: <name> (code <code>), leaving out the code part when unsure. Nothing else."*

System One screening question. The state is the task instruction alone (task-only) or the agent's trajectory; the options are Yes and No: *"Consider this order for the patient: [order]. Is this order indicated for this patient as part of completing the task (whether or not the agent has already placed it)?"* The score is P(Yes).

Checklist at the finish. *"Before you finish: a screening model reviewed orders that came up while working on this task and flagged these as possibly indicated for this patient and not yet placed: [eight orders]. Check each against the chart and the task. Place any that are indicated, and only those, then finish."*

Restart at the finish (full and fresh context). *"Based on the task and the patient data gathered so far, place every order that this task requires and that has not been placed yet, then finish."* In the fresh arm, every assistant turn that calls an order, message, appointment or note tool, and the tool results answering it, are removed; the system prompt, the task and all retrieval turns remain. Only order calls are executed, minus repeats of placed orders.

First order step. Reminder: *"Reminder of your task: [task instruction]"*. Plan first: *"Before placing any orders, write out your full assessment and plan in your reply, listing every order the task requires. Then place every order in that plan."*

Order appropriateness (judges). The judge sees the task and the retrieved chart (the fresh context above) and one candidate order, and answers one letter: A indicated (standard of care for the patient's current problem); B reasonable (a defensible addition a careful physician might order); C low value (unnecessary, not supported by the data, or unrelated to the task); D potentially harmful (contraindicated, a wrong drug or dose for this patient, or risk clearly exceeding benefit). Thinking on, temperature 0. The pre-registered version rated each case's orders in one list; rated in a list, near-duplicate variants of one order sit side by side and later copies are marked low value, so we report the single-order ratings in the main text, and the list version (in which no checklist order is rated harmful either) in the research log.

C Pre-registered tests

Every confirmatory test was written into a dated research log, with its prediction and bar, before any of its data existed; amendments were logged before results were seen. Table 2 lists them in the order run. Tests that failed shaped the study as much as those that held: P1, P9 and O1 moved us from independent runs to shared-prefix branching, the unplaced-order breakdown of P6 led to the task-only screen, and P8b and P10 kept the agent's orders out of the ranker.

Test	Prediction	Result	Verdict
P1	pplx12's S1-SELECT gain on required orders replicates on 24 unseen tasks	-18 points $[-36, -2]$	fails
P2′	orders-only gate: required-order gain, notes at the LLM's level	+5 $[-2, 14]$; notes, steps restored	primary fails
P3	a consequential exam key ranks deciders as online	11 questions, too few	not supported
P4	blind decider picks like the informed one (style only)	agreement 41%	rejected
P5	gate places more orders, more outside the rubric (replication)	+0.61 [0.12, 1.13]; +0.35 n.s.	half holds
P6	a System One screener reading the agent's state ranks required orders	AUROC 0.74 (unplaced: 0.43, post hoc)	holds
P6b	a richer chart helps on the unplaced orders	0.44	fails
P6c	a task-only screener ranks the unplaced orders	0.65 [0.51, 0.79]	holds
P7′	in-run checklist beats a frequency-ranked one	-0.02 $[-0.15, 0.11]$ (interim)	fails
P8	the task-prior pool covers the open orders	0.84 [0.71, 0.95]	holds
P8b	showing placed orders to the ranker helps	top-3 0.31 $\rightarrow$ 0.15	fails
P9	task-prior checklist beats a random one (independent runs)	+0.045 $[-0.07, 0.19]$	fails
D1	no drift between old and new LLM runs	-0.004 $[-0.09, 0.08]$	holds
G1	task-only beats trajectory for System One models†	+0.17 to +0.34, 6/6	holds
G4	one fabricated order raises P(Yes) by $\geq$0.15	+0.07 to +0.10	fails on size
G5	removing decisions recovers the checker (System One)	+0.21 to +0.25, 5/5	holds
G5-LLM	the same for Qwen	+0.12 $[-0.005, 0.26]$	point holds
G6	the orders, not the notes, anchor	shares 0.57–0.83 vs $\approx$0	holds (4/5)
G6-LLM	the same for Qwen	no anchoring on its answered set	not testable
G7	a debiasing instruction does not undo it (also Qwen)	1–8% of the gap	holds
P10	factorized ranking with a repeat judge keeps coverage	top-3 0.15 vs 0.31	fails
O1	checklist beats an independent baseline run	-0.02, n.s.	fails (noise)
A1, A2	nudge / fresh ask at the finish (offline)	+0.08, +0.07 (bars 0.15, 0.10)	fail on size
A3, A4	reminder / plan at the first order step (all required orders)	+0.11, +0.19	hold
A5	most noted misses are false completion claims	0.40	fails
R1	fresh $-$ full restart (Qwen)	+0.071 [0.010, 0.137]	holds
R2	full restart $-$ finish (Qwen)	+0.080 [0.026, 0.151]	holds
R3	checklist $-$ finish; $-$ random list (Qwen)	+0.147; +0.059 [0.006, 0.118]	holds
R3-d3	the same with d3 as ranker	+0.160; +0.072 [0.011, 0.133]	holds
XA	plan $-$ none; plan $-$ reminder (Gemma, finally missed orders)	+0.177; +0.114 [0.022, 0.228]	holds
XR1, XR2	fresh $-$ full; full $-$ finish (Gemma)	+0.089; +0.178	hold
XR3	checklist $-$ finish; $-$ random list (Gemma)	+0.292; +0.136 [0.065, 0.217]	holds
XQ3	Gemma's error types match Qwen's	1 of 3 types	fails
H	checklist orders not less appropriate than the agent's; none harmful	harmful 0; others 1 of 2 agents	partial
Harms	orders outside the rubric, checklist $-$ finish $\leq$ +1.0	+1.47 (Qwen), +1.56 (Gemma)	fail

Table 2: **The pre-registered tests, in the order run.** Intervals are 95% bootstrap over tasks or cases; online outcomes are open required-order checkpoints unless stated; offline screens report AUROC. Forbidden orders were 0 in every arm. †G1 was fixed after pplx12 and Clef were scored and before the other four models were. The harm bound on orders outside the rubric fails for every checklist; §7 shows what those orders are.

D The decision seat in detail

This appendix gives the full study of System One models in the agent's decision seat summarised in §3.

D.1 Putting a System One model in the decision seat

An agent loop alternates between a state s_t, the task plus every action and result so far, and the next turn a_t: a set of tool calls with arguments, or a final answer. The benchmark samples $a_t \sim \pi(\cdot \mid s_t)$ from the LLM. We replace only this step.

LLM (reference). One sample from π with thinking on and the model's default sampling, exactly as the benchmark prescribes. A logging variant samples $K{=}5$ turns and executes the first, which has the same distribution as LLM and gives the run-to-run noise floor.

S1-SELECT: the System One model chooses the turn. The LLM samples $K{=}5$ turns $a_t^{(1)}, \ldots, a_t^{(5)}$ i.i.d. from π in one request. Exact

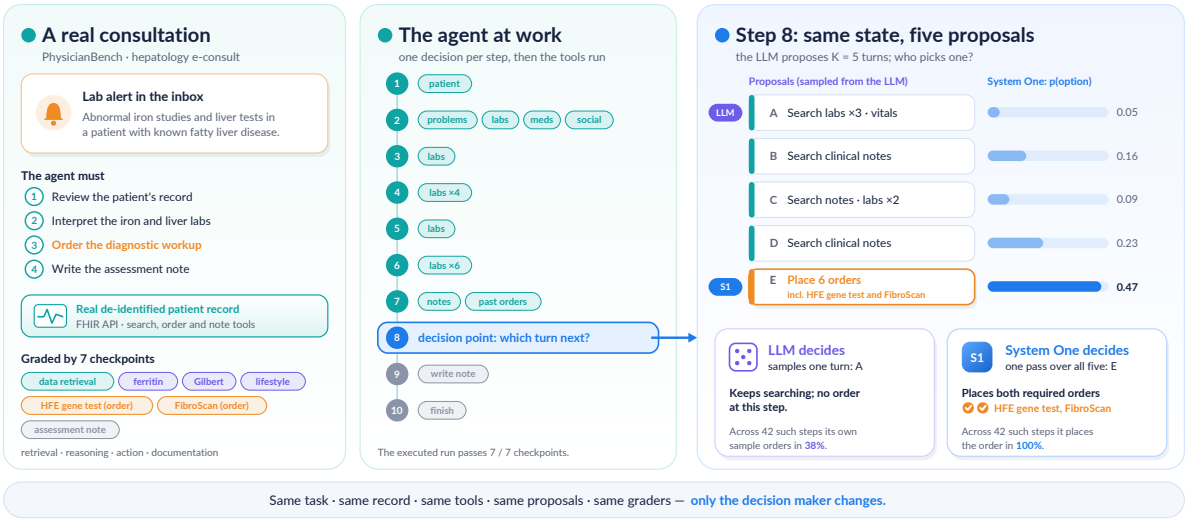


Figure 5: **One real consultation, one decision, two deciders.** A PhysicianBench task (instruction paraphrased): a hepatology e-consult on abnormal iron and liver tests, in which the agent must review the record, interpret the labs, order the workup and write a note, graded by seven checkpoints. At step 8 the LLM has proposed five next turns. Its own sample (A) keeps searching labs. The System One model reads the same state and all five proposals in one forward pass and puts the most probability on E, which places both orders the graders require. Across all 42 order steps where a proposal places a required order, the LLM's own sample orders in 38% and Clef in 100%; most of the LLM's misses are placed at a later step (§D.3). The figure shows tool types, probabilities and outcomes only, never patient data.

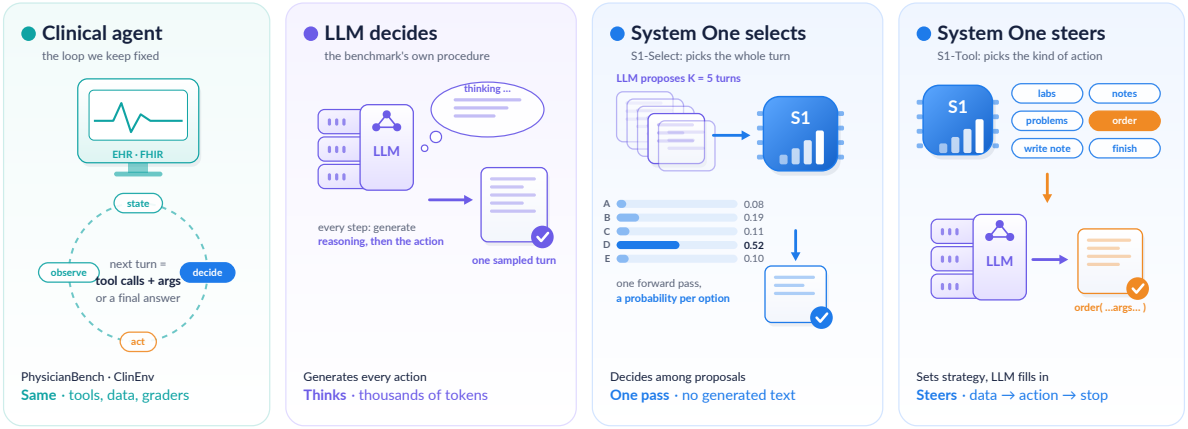


Figure 6: **Ways to make the next decision in the same clinical agent loop.** The loop, tools, records and graders stay fixed. LLM: the model samples its next turn after reasoning. S1-SELECT: the LLM proposes five complete turns and a System One model picks one in a single forward pass. S1-TOOL: the System One model picks the kind of action from the full tool menu and the LLM writes the call. A gated variant of S1-SELECT (§D.1) consults the System One model only at order steps.

duplicates are merged. The System One model reads the state and the distinct turns, rendered as tool calls with full arguments, and returns a probability for each; the most probable turn is executed. It never sees the LLM's reasoning, so it judges actions, not rationales. A uniform pick among K i.i.d. samples has the same distribution as LLM, so any difference between the two comes from *which* sample is chosen.

S1-TOOL: the System One model chooses the kind of action. The System One model reads the state and the full tool menu (each tool's name and description, plus FINISH) and picks one. The LLM then writes the turn with its calls constrained to the chosen tool. The System One model now sets the agent's strategy and timing, and the LLM supplies the content. LLM-TOOL is the control: the same one-tool-per-turn restriction with the LLM choos-

ing (it keeps only the calls of its own turn's first tool).

Gated selection: System One plus the reasoning LLM. The System One model is consulted only at some steps and only overrules the LLM's own first sample there. In the variant we test online it is consulted only where at least one proposal places a service or medication order; elsewhere, including every choice between finishing, writing the note and gathering more data, the LLM's first sample is executed. A margin variant overrules only when the System One model prefers its pick to the LLM's sample by a log-odds margin.

Fair inputs. Options are named by letters shuffled with a seed per task, run and step. The System One model reads a deterministic 32k-token view of the state (oldest tool results shortened first); the LLM reads up to 128k tokens. If the System One model fails, the first sample is executed; this happened 0 times in the S1-SELECT run.

D.2 Performance: choosing among proposals

Table 3 and Figure 7 give the headline for Clef. S1-SELECT passes 80.2% of checkpoints against 81.3% for the LLM (Δ −4.0 [−13.0, +5.0] points in task success, McNemar $p = 0.52$). The noise floor is wide: two runs of the same LLM policy differ on 28 of 100 tasks (Δ +0.0 [−10.0, +10.0] points), so single-run differences in task success of a few points carry no signal. Pooling the LLM's runs sharpens the checkpoint-level comparison (Table 4).

Neither System One model moves the checkpoints. An early gain of pplx-decider-v1.2 on required orders failed a pre-registered replication (§D.3). Both shorten the episode: they act after less retrieval, write fewer notes and take fewer steps. Documentation checkpoints dip by 2–4 points, within noise. Nothing in our data establishes a gain in task success.

D.3 Deciding is not the bottleneck

Why does a better pick not show up in the outcome? On the fixed exam every decider makes far fewer key errors than the LLM's own sample (0.17–0.27 against 0.48; §D.4), yet online the checkpoints do not move. Following each required order along the LLM's own trajectories resolves the gap (Table 1). Of the 94 steps at which the LLM's sample skipped an available required order, 79% are followed by that order later in the same run: the step-level key mostly rewards ordering early, not ordering at all. Of all 118 required orders in 54 tasks, the LLM places 69%, 11% are proposed at some step but never executed, and 19% are never proposed by any sample. Selection can at most recover the 11% (on these trajectories, 4 more of 54 tasks would pass); the 19% are bounded by generation. The decision seat holds little of the LLM's error.

pplx-decider-v1.2 shows what a selector does with that margin. It recovers part of it (proposed but never executed: 4% against 11%, −7 points [−13, 0]) and gives about as much back on generation: it acts after less retrieval, and 4 points more of the required orders are never proposed ([−5, 13]). It places about as many as the LLM (+3 [−8, 12]). Its apparent gain on its first 76 tasks (+13 points on required-order checkpoints) failed a pre-registered replication on the 24 tasks it had not run (−18 [−36, −2], against an LLM that already passes 84% of them).

The exam cannot see any of this. Restricted to steps where the counterfactual is known (the LLM's sample takes no open required order and a proposal places one the run never places), it holds only 11 questions in 9 tasks, too few to rank deciders (take rates 0.64–0.91, intervals ±0.2–0.3). Step-level answer keys inside an agent's own trajectory measure decision *style*; only whole runs rank deciders on outcome.

D.4 Mechanism: a commitment bias

Why do the System One models shorten the episode? The fixed exam answers on identical inputs (Tables 9 and 5). All six models agree with the LLM's own sample about as often as a random pick would (21–27% for five of them, 36% for linne-oak-1.0; random 22%), and with each other more often. They make far fewer key errors than the LLM's sample (0.17–0.27 against 0.48; every bootstrap interval excludes 0), but the errors move rather than vanish: the LLM omits an available required order at 55% of such steps and the deciders at 2–18%, while the deciders order when no required order is on offer at 52–71% of steps against the LLM's 35%, and finish while a required order is open at 90–97% of such steps against 80%.

Two observations show that much of this is a preference over kinds of action rather than a reading of the patient. First, a rule with no model, *always take an ordering turn when one is offered*, reaches an error of 0.22 and leaves no required order behind; on the order key it is better than every

	Task pass@1		All checkpoints		Checkpoints by type (%)					
Decision maker	%	Δ [95% CI]	%	Δ [95% CI]	DR	CR	AE	Doc	Calls /task	Tokens /task
LLM	**41.0**	*ref.*	**81.3**	*ref.*	83.2	**84.1**	75.0	**70.7**	36.3	29k
S1-SELECT	37.0	−4.0 [−13.0, +5.0]	80.2	−1.1 [−4.7, +2.6]	**85.0**	82.8	**80.1**	65.7	31.2	119k
S1-TOOL	7.0	−34.0 [−45.0, −23.0]	44.3	−37.0 [−44.5, −29.3]	44.7	47.5	52.3	26.3	25.7	18k

Table 3: **PhysicianBench, run 1, 100 tasks completed by all three decision makers** (Clef as the System One model). Δ: difference from LLM in points with a 95% task bootstrap interval. Checkpoint types: data retrieval (DR), clinical reasoning (CR), action execution (AE), documentation (Doc). Calls: tool calls per task. Tokens: generated tokens per task, including all five proposals for S1-SELECT. Bold: best per column.

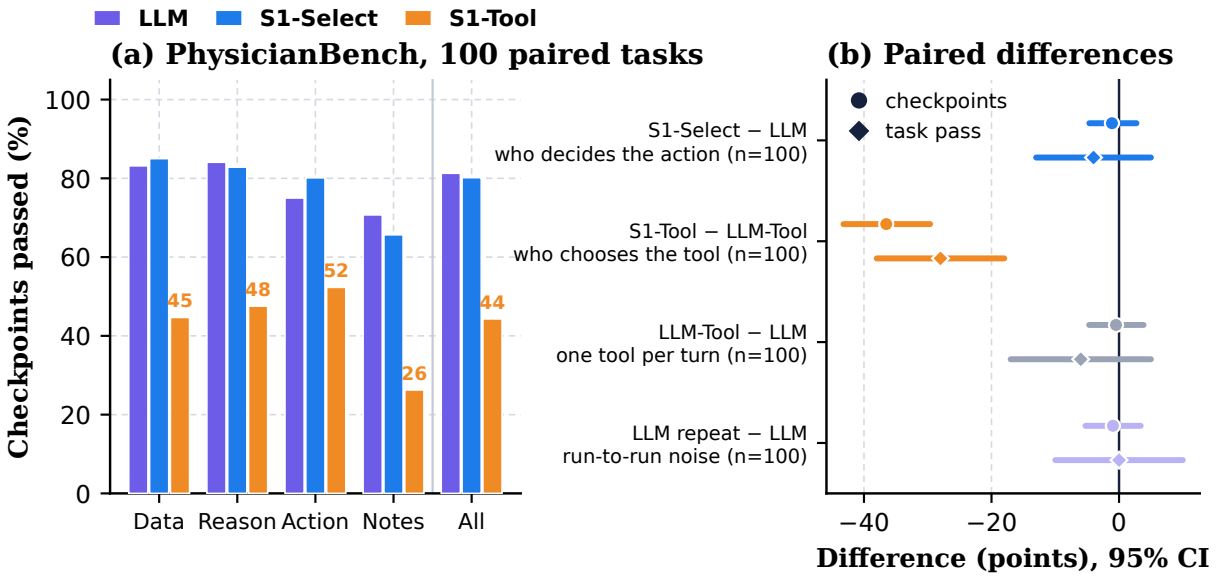


Figure 7: **A System One model keeps the LLM's quality when it chooses among proposals, and falls behind when it steers.** (a) Checkpoint success by type on the paired tasks. (b) Paired differences with 95% intervals, each on the tasks both arms completed.

	LLM (pooled)	Clef (100 tasks)	pplx-decider (100 tasks)
Task pass	0.41	−0.04 [−0.12, 0.03]	+0.04 [−0.06, 0.14]
Required-order cp.	0.76	+0.01 [−0.06, 0.07]	+0.04 [−0.05, 0.14]
Documentation cp.	0.70	−0.04 [−0.11, 0.03]	+0.03 [−0.05, 0.12]
Required orders placed	0.68	−0.02 [−0.10, 0.05]	+0.05 [−0.05, 0.15]
Notes written	1.5	**−0.38** [−0.60, −0.09]	**−0.52** [−0.68, −0.37]
Retrieval before 1st action	30	**−5.5** [−8.0, −3.1]	**−3.2** [−5.6, −1.0]
Steps	36	**−5.1** [−8.1, −1.9]	**−4.0** [−6.7, −1.5]

Table 4: **S1-SELECT against the pooled LLM runs**, paired by task (differences with 95% task bootstrap intervals; LLM column: the level on the 100 paired tasks). cp.: checkpoint pass rate. Bold: interval excludes 0.

decider. Second, the deciders' low redundancy is not evidence that they track what has been done: in 17–18 of the 19–22 questions where a decider avoids the LLM's redundant option, it does so by choosing FINISH over a rewrite of the note. Table 6 puts the preference in one number per kind: every model picks ordering turns 1.3–1.6× and finishing turns 1.3–1.4× as often as the LLM's own sample does, and retrieval turns 0.89–0.93×. The same profile shows online (Table 4: less retrieval before acting, fewer notes, shorter runs). We call it a *commitment bias*: System One deciders prefer turns that commit (order, write, finish) over turns that gather information.

The bias is useful exactly where the LLM undercommits, and it costs elsewhere. pplx-decider-v1.2 online takes more of the required orders that a proposal offers (offered but never executed: 4% of required orders, against 11% for the LLM; difference −7 points [−13, 0]), but 4 points more are never proposed at all ([−5, 13]), so it places about as many as the LLM (+3 [−8, 12]). Clef does not even gain the first part: on its own trajectories it places 6 points fewer of the required orders than the LLM ([−15, 3]), and 8 points more of them are never proposed at all ([−1, 18]), plausibly because it moves to action after less retrieval and the LLM then never thinks of them.

	Error (3 keys)	Omits required	Over-orders	Finishes early	Redundant
LLM (own sample)	0.48	55	35	80	28
Always order (no model)	0.22	0	100	77	25
Clef-27B	0.19	2	66	90	8
pplx-decider-v1.2	**0.17**	3	71	90	4
d3	0.18	4	67	90	4
linne-oak-1.0	0.21	5	53	97	9
amos-1-large	0.27	18	53	90	11
decider-31b	0.21	7	52	93	8

Table 5: **Error profile on the fixed exam** (1,460 questions answered by all six, 100 tasks; 200 keyed: 97 order, 30 finish, 85 redundancy). Error: bad picks over all (question, key) pairs. Other columns in %: omits a required order on offer; orders when none is required among mixed options; finishes while a required order is open; picks a redundant option. Always order: take the most-sampled ordering option whenever one is offered.

	Order	Finish	Note	Retrieve
Clef-27B	1.57	1.34	1.06	0.89
pplx-decider-v1.2	1.49	1.35	1.09	0.90
d3	1.48	1.37	1.09	0.90
linne-oak-1.0	1.37	1.43	1.18	0.89
amos-1-large	1.29	1.30	1.06	0.93
decider-31b	1.36	1.41	1.18	0.89

Table 6: **The commitment bias.** Picks of each kind of turn on the exam, relative to the LLM's own sample on the same questions (> 1: chosen more often than the LLM chooses it).

A prior and a misreading (blind test, preregistered). Does the bias read the patient at all? We asked pplx-decider-v1.2 the whole exam again with the options but no state (same option letters; test P4, fixed before its data existed). The preregistered style hypothesis, that blind and informed picks agree on at least 60% of questions and the order and finish errors move by at most 0.05, is rejected: they agree on 41% (random 22%), and blind picks match the kind of the informed pick barely more often than chance (91% against 88%; other deciders with the state: 95–98%). The model reads the state. But the keys move in opposite directions (Table 7). On orders, the blind model is already far better than chance: about two thirds of the reduction from a random pick needs no patient state, so the order preference is largely a prior over what the options look like. On redundancy, the state does most of the work. On finishing, the state hurts: these are late steps where 73% of the options are FINISH and the LLM itself mostly wants to stop. Blind, the decider avoids finishing (below chance); with the state, it finishes while a required order is open more often than chance and more often than the LLM's own sample. The premature finishing is thus a misreading of the state, which agrees with, and strengthens, the LLM's own sense that the case is done.

Key error	Random	Blind	State	Blind − state
Order (97 q.)	0.55	0.22	0.07	+0.14 [0.08, 0.21]
Redundancy (86)	0.35	0.24	0.03	+0.21 [0.11, 0.31]
Finish (30)	0.73	0.53	0.90	−0.37 [−0.53, −0.19]

Table 7: **Blind test (P4), pplx-decider-v1.2.** Key errors of a random option, of the decider without the patient state (options only) and with it; differences with task-bootstrap 95% CIs. The LLM's own sample: 0.55, 0.28, 0.80.

Tool	Clef (S1-TOOL)	LLM (LLM-TOOL)
Write note	44	7
Search labs	17	31
Search notes	10	20
Search problems	4	9
Place order	7	4
Search med. orders	3	7
Demographics	5	5
Finish	4	4

Table 8: **Which tool each decider picks** under the same one-tool-per-turn restriction, as a share of steps (%), on the tasks both completed.

D.5 Steering alone, the bias breaks the agent

Given the tool menu instead of proposals, the same model fails. S1-TOOL passes 7.0% of tasks and 44.3% of checkpoints (Δ −34.0 [−45.0, −23.0] points in task success, $p < 0.001$). It retrieves 14.2 times per task against the LLM's 31.2, starts acting after 9.9 retrieval calls instead of 30.1, and writes the note 9.2 times, rewriting it again and again; it chooses to write the note at 44% of steps, where the LLM does so at 7% (Table 8). Restricting the LLM to one tool per turn costs nothing measurable (LLM-TOOL versus LLM: −0.5 [−4.7, +3.9] checkpoint points), while handing the same restricted choice to the System One model costs −36.5 [−43.2, −29.6] points. pplx-decider-v1.2 in the same seat falls behind as well (−24.7 [−34.7, −14.8] points over 49 tasks). In an open action space the commitment bias has nothing to anchor it: committing means writing, and the agent writes instead of looking.

D.6 Boundaries: where a System One decider is reliable

On the keyed exam questions we split the benefit of executing the decider's pick instead of the

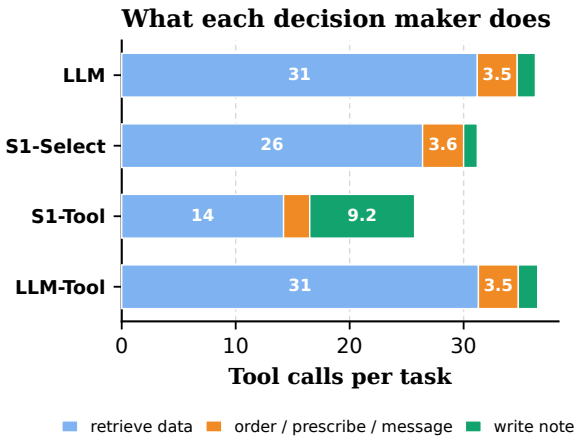


Figure 8: **Tool calls per task by kind.** Steering, the System One model under-retrieves and over-documents. LLM-TOOL: the LLM under the same one-tool-per-turn restriction, on the tasks it shares with S1-TOOL.

LLM's sample (key errors avoided per question) by what the decider and the LLM signal (Clef and pplx-decider-v1.2 shown; the other models behave alike).

Its confidence is informative. Overrides where the decider prefers its pick to the LLM's sample by a log-odds of at least 3 avoid 0.8–1.0 order-key errors per question and 0.6 redundancy errors; overrides below 1.5 avoid almost none. A margin keeps most of the benefit with far fewer overrides.

It helps where the reasoning model is unsure. When only 1–2 of the LLM's five samples are semantically the same decision as its first sample (same tools and key clinical arguments), the decider avoids 0.37–0.40 errors per question; at 4 of 5 it avoids 0.11–0.17, and at 5 of 5 none. Order decisions sit almost entirely at 1 of 5. The LLM's self-consistency and the decider's confidence are complementary signals for when to hand over.

Finishing is its blind spot. Where the decider confidently overrules the LLM's sample toward FINISH while a required order is still open, it is wrong in 4 of 5 cases (error change -0.75 to -0.80, $n = 4$–5), and every model finishes early more often than the LLM (Table 5). In S1-TOOL the same blindness appears as an agent that never stops writing.

Long states. 85% of keyed questions exceed the decider's 32k window and are read with shortened tool results. The deciders' error is 0.03–0.07 on the questions read in full and 0.20–0.23 on shortened ones, against 0.44–0.47 and 0.51–0.52 for the LLM's sample; long, truncated trajectories are where the System One view is weakest (confounded with late steps).

	Order steps (answer key)			All questions		
Decider	Right	Orders, required	Orders, none req.	Agrees w/ LLM	Act	Note
Random	45	–	–	22	–	–
LLM (own sample)	45	45	35	100	–	–
LLM (most sampled)	45	–	–	–	–	–
Clef-27B	94	98	66	23	15	10
pplx-decider-v1.2	93	97	71	27	14	11
d3	91	96	67	21	14	11
linne-oak-1.0	92	95	53	37	13	11
amos-1-large	78	82	53	22	12	10
decider-31b	91	93	52	24	13	11

Table 9: **The same exam for every decider** (PhysicianBench, the LLM's own logged states and proposals; 1461 questions answered by every decider, 97 of them with an order key). Right: picks a required order on order steps where options differ. Orders, required / none req.: picks an ordering option when one places a required order, or when none does. Agrees w/ LLM: same pick as the LLM's own sample. Act, Note: share of all picks that place orders or write the note.

	S1-SELECT vs LLM		S1-TOOL vs LLM-TOOL	
Decider	Tasks	Δ checkpoints	Tasks	Δ checkpoints
Clef-27B	100	−1.1 [−4.7, +2.7]	100	−36.5 [−43.2, −29.6]
pplx-decider-v1.2	76	+1.1 [−5.0, +7.2]	49	−24.7 [−34.7, −14.8]

Table 10: **Whole tasks with another decider in Clef's seat.** Paired checkpoint differences with 95% intervals, each on the tasks both arms have completed.

D.7 The same behaviour across six models

Is this Clef's behaviour, or how System One models behave as deciders? Table 9 gives every model's answers to the fixed exam. On order steps all six pick a right option far more often than the LLM's own sample (Clef 94%, pplx-decider-v1.2 93%, d3 91%, linne-oak-1.0 92%, decider-31b 91%, amos-1-large 78%; the LLM 45%, random 45%), and all of them also order when no required order is on offer (53–71% of such steps, the LLM 35%). They agree with each other more than with the LLM (pplx-decider-v1.2 and d3 on –% of questions, Clef and pplx-decider-v1.2 on 50%). Online, pplx-decider-v1.2 in S1-SELECT keeps the LLM's checkpoint level (+1.1 [−5.0, +7.2] points over 76 tasks) and in S1-TOOL falls far behind (−24.7 [−34.7, −14.8] points over 49 tasks; Table 10). The commitment bias, its benefit on orders and its failure when steering are shared by the models we could test, not a quirk of Clef.

D.8 System One plus the reasoning LLM

The boundaries suggest a narrower role. Let the LLM gather data, write the note and decide when to stop; consult the System One model only where a proposal places an order, the decisions it gets right and the LLM defers. On the exam, this *orders-only* gate keeps the decider's order behaviour (pplx-decider-v1.2: omission 3%) and leaves finishing and redundancy at the LLM's level, while overruling the LLM on 11% of steps instead of 73%. Gating at every order or finish step overrules on 19%; a gate by log-odds margin or by the LLM's self-consistency does not lower the exam error further under cross-validation, because the exam's redundancy key rewards finishing over a note rewrite and the keys pull against each other. The exam cannot choose among these designs; whole runs must.

Pre-registered online tests. We fixed these tests before their data existed. **P1** replicated pplx-decider-v1.2 in S1-SELECT on the 24 tasks it had not run; prediction: more required-order checkpoints than the pooled LLM runs. It failed (-18 points [-36, -2], 16 tasks with such checkpoints; §D.2). **P2′** ran the orders-only gate with pplx-decider-v1.2 on all 100 tasks; prediction: the required-order gain of S1-SELECT (which P1 has since failed to replicate) without its loss of notes (notes within the LLM's run-to-run noise); documentation not lower; task pass claimed only if its CI excludes 0. The primary prediction fails: required-order checkpoints $+5$ points [-2, 14]. Table 11 pools this run with a second one (P5, below). The secondary ones hold: notes and steps return to the LLM's level, and documentation is not significantly lower. Task pass does not change. The design does what the blind test (P4, §D.4) predicts for its structure: away from finishing, the decider no longer stops the agent early. What it cannot do is pass the ceiling of §D.3.

The gate over-orders. In the P2′ run the gated agent placed 1.3 more orders per task than the LLM [0.76, 1.89], nearly all matching no order in the rubric [$+1.13$, 0.52 to 1.79]. We saw this after the fact and pre-registered a replication, **P5**: a second gate run, predicting more orders (at least $+0.5$, CI excluding 0) and more orders outside the rubric (CI excluding 0). The first half replicates ($+0.61$ [0.12, 1.13]); the second does not reach significance in the second run alone ($+0.35$ [-0.29, 0.97]). Pooled over both runs (Table 11) the gate places 0.95 more orders per task, 0.74 of them outside the rubric. Full steering does not show this, plausibly because it ends runs earlier and so gives the order prior fewer steps to act on; the gated agent runs as long as the LLM, and at every order step the decider prefers the ordering turn. This is the state-free part of the commitment bias, now visible as volume. PhysicianBench barely penalises it (one task forbids an order), but in practice extra orders cost money and can harm patients. An exploratory gate that also consults the decider at finish steps (margin 1.5, 22 tasks; not part of any test) brings the losses back: notes -0.62 [-1.02, -0.29], steps -6.7 [-12.9, -1.9]. The harm of steering sits at the finish step, as the blind test predicts.

Table 11: Online, all 100 tasks: the orders-only gate (two runs, pooled) and full steering in S1-SELECT (one run), both with pplx-decider-v1.2, each minus the pooled LLM runs, with task-bootstrap 95% CIs; the last column is the difference between two LLM runs (run-to-run noise). Required orders: the 54 tasks with such checkpoints.

	gate − LLM	S1-SELECT − LLM	LLM − LLM
required-order checkpoints	+0.06 [−0.01, 0.14]	+0.04 [−0.05, 0.14]	–
required orders placed	+0.04 [−0.03, 0.12]	+0.05 [−0.05, 0.15]	+0.05 [−0.01, 0.11]
orders placed per task	**+0.95** [0.51, 1.44]	+0.26 [−0.35, 0.93]	+0.18 [−0.34, 0.71]
of which not in the rubric	**+0.74** [0.20, 1.31]	+0.21 [−0.70, 1.11]	+0.35 [−0.31, 1.02]
notes written per task	−0.07 [−0.26, 0.12]	**−0.52** [−0.68, −0.37]	−0.12 [−0.40, 0.15]
steps per task	−0.5 [−3.9, 3.2]	**−4.0** [−6.7, −1.5]	–
Documentation checkpoints	−0.03 [−0.09, 0.04]	+0.03 [−0.05, 0.12]	–
task pass	−0.01 [−0.09, 0.06]	+0.04 [−0.06, 0.14]	–

E Additional results on the decision seat

Table 12 lists every paired comparison with intervals; Table 13 describes the decisions. Episodes that ended without a final answer (step limit, repeated-call abort or context overflow) number 2 for LLM, 1 for S1-SELECT and 16 for S1-TOOL on the paired tasks. Distinct proposals are semantically different decisions at 89% of S1-SELECT steps; on the exam, 952 of 1,460 questions are choices among retrieval turns only, which no key scores.

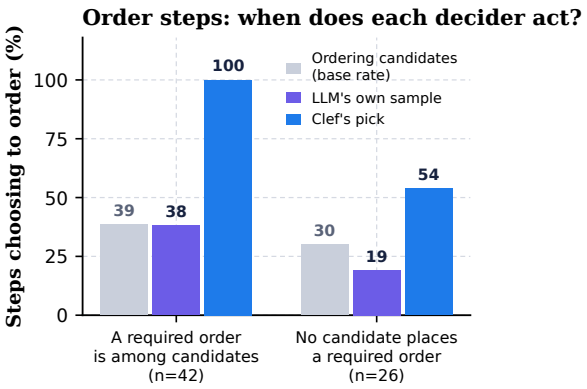


Figure 9: **Clef places required orders when they are on the table.** Order steps of the S1-SELECT run whose proposals mix ordering and other turns. Orders the graders do not require are not wrong, only uncredited; most of the LLM's skipped orders are placed at a later step (§D.3).

Comparison	Tasks	Pass@1	Δ pass@1 [95% CI]	p	Δ checkpoints [95% CI]	Δ DR	Δ CR	Δ AE	Δ Doc
S1-SELECT vs LLM	100	37.0 / 41.0	−4.0 [−13.0, +5.0]	0.52	−1.1 [−4.7, +2.7]	+1.8	−1.2	+5.1	−5.1
S1-TOOL vs LLM-TOOL	100	7.0 / 35.0	−28.0 [−38.0, −18.0]	<0.001	−36.5 [−43.2, −29.6]	−39.3	−36.9	−27.4	−38.4
LLM-TOOL vs LLM	100	35.0 / 41.0	−6.0 [−17.0, +5.0]	0.38	−0.5 [−4.7, +3.9]	+0.8	+0.4	+4.7	−6.1

Table 12: Paired comparisons, each on the tasks both arms completed (run 1). Δ in points; per-type columns give the point difference in checkpoint success.

	Steps /task	Distinct strings (%)	Distinct actions (%)	Mean max p	Norm. entropy	Departs from LLM sample (%)	Decide (s)
S1-SELECT	11.8	99	89	0.382	0.886	79	4.7

Table 13: S1-SELECT decisions, run 1. Distinct strings: steps whose five proposals are not all identical. Distinct actions: after merging proposals with the same tools and key clinical arguments. Mean max p and entropy (normalised by $\log n$) describe the System One model's distribution where there was a choice. Departs: share of those steps where it chose a proposal other than the LLM's first sample.